A Formal Specification of `model-provider` (target state: a routing tool, multi-source)

Punt Labs

October 10, 2026

1 Purpose and scope

This is the *target* specification — the robust design a from-scratch re-implementation targets — paired with `model-provider.current.spec.tex`, which models the tool as built today.

The organising idea, reached by separating two axes the combined tool conflated: *routing* (how a request reaches an upstream — base URL, auth, which source) is session-global per harness; *model-choice* (which model serves a request) is per-agent, user-authored, and out of scope. `model-provider` owns routing. Its whole job is: the operator picks, per harness, a *source* to route through and a default model, and the tool writes exactly that route so the chosen model — and any model any agent later names — reaches the upstream the operator picked.

The user picks the route. There is no tool-defined precedence and no ban on overlap: if two sources both serve a provider, the operator simply routes a given harness through whichever source they prefer. *That choice is the entire tool.* The managed state is therefore a per-harness map from harness to source, set by the operator, never derived.

Multiple sources. One operator may have a single source (a gateway like Bifrost that serves every provider and carries both protocols); another may have several narrow sources (direct OpenAI, OpenRouter) and no gateway at all. The spec models a *set* of configured sources and a per-harness route, so the single-gateway case is just the degenerate "every harness routed through one source that serves everything."

2 Given sets and enumerations

A model id is an opaque catalog token (e.g. `openrouter/anthropic/claude-opus-4.8`); its leading segment names a provider. A provider is the upstream a source routes to (`anthropic`, `openai`, `bedrock`, `openrouter` as an aggregator). A source is a configured route target: a gateway (Bifrost, Envoy, AgentGateway) or a direct vendor/aggregator endpoint.

$[ModelId, Provider, Source]$

The four harnesses.

$$Harness ::= claudecode \mid codex \mid pi \mid opencode$$

The protocol each harness speaks. Claude Code speaks Anthropic; the other three speak OpenAI. A source must expose a harness's protocol to carry it.

$$Protocol ::= anthropic \mid openai$$

Auth modes a source may accept: a per-host virtual key / API key (*vkauth*), or the operator's Claude subscription OAuth (*subauth*).

$$AuthMode ::= vkauth \mid subauth$$

Claude Code's built-in model-alias tiers — the Agent tool's `opus|sonnet|haiku|fable` knobs. Not user agent definitions; built-in alias slots the tool sets so a model-less spawn resolves to a routable id. A fixed, finite set.

$$Tier ::= opus \mid sonnet \mid haiku \mid fable$$

3 Harness and model properties

Which protocol each harness speaks.

$$speaks : Harness \longrightarrow Protocol$$

$$\begin{aligned} speaks = \{&claudecode \mapsto anthropic,\\ &codex \mapsto openai,\\ &pi \mapsto openai,\\ &opencode \mapsto openai\} \end{aligned}$$

Which harnesses canonicalize the configured id (strip the provider prefix), losing the routing information unless an override restores the verbatim id. Only Claude Code.

$$canonicalizes : \mathbb{P}\ Harness$$

$$canonicalizes = \{claudecode\}$$

Which harnesses present a native model picker that validates against a tool-written list (so the tool must populate it). pi and opencode do; codex and Claude Code do not.

$$showsPicker : \mathbb{P}\ Harness$$

$$showsPicker = \{pi, opencode\}$$

Every model id carries a provider — the only thing routing depends on, since a source fans out by provider. Total, so there are no partial-application gaps.

$$providerOf : ModelId \longrightarrow Provider$$

4 The source contract

For each source: *srcServes* is the set of providers this account is entitled to reach through it (a gateway serves many; a direct source one or few; the Bedrock Anthropic-models entitlement gap is one such restriction). *srcProtocols* is the protocols it exposes; *srcAuth* the auth modes it accepts. Every source serves at least one provider, exposes at least one protocol, and accepts at least one auth mode.

$$
\begin{array}{|l}
srcServes : Source \longrightarrow \mathbb{P}\ Provider \\
srcProtocols : Source \longrightarrow \mathbb{P}\ Protocol \\
srcAuth : Source \longrightarrow \mathbb{P}\ AuthMode \\
\hline
\forall\, s : Source \bullet \\
\quad srcServes\ s \neq \varnothing \land \\
\quad srcProtocols\ s \neq \varnothing \land \\
\quad srcAuth\ s \neq \varnothing
\end{array}
$$

The universe contains at least one *Anthropic-capable* source with a routable model and both auth modes — a witness, not a requirement on any particular configuration. Its sole purpose is to force the model checker's constant search to admit a source through which Claude Code can route, so the Claude-Code guarantees (override, four-alias routability, the *subauth* path) are exercised on reachable states rather than passing vacuously. Operators with no Anthropic source simply never route Claude Code; this witness does not oblige them to.

$$
\begin{array}{|l}
anthropicWitness : Source \\
anthropicModel : ModelId \\
\hline
anthropic \in srcProtocols\ anthropicWitness \\
openai \in srcProtocols\ anthropicWitness \\
srcAuth\ anthropicWitness = \{vkauth, subauth\} \\
providerOf\ anthropicModel \in srcServes\ anthropicWitness
\end{array}
$$

The picker pattern: the set of ids the operator's picker expression matches (a regex in the implementation; its match set here). The native picker offers the pattern intersected with what is routable on that harness's source — a view, never a constraint on what may be routed.

$$
\begin{array}{|l}
matchesPattern : \mathbb{P}\ ModelId
\end{array}
$$

5 Configuration state

The managed state is routing, per harness, plus the fixtures the one canonicalizing harness needs. *configured* is the set of sources the operator has set up. *routeVia* maps each *routed* harness to the source the operator chose for it; its domain is exactly the harnesses currently on a managed route (others are

on their vendor default / direct). *model* gives each routed harness its default model; *hauth* its auth mode. *override* records which canonicalizing harnesses have the `modelOverrides` fixture written. *aliases* gives Claude Code's four tier knobs. *shown* gives, for each picker-bearing routed harness, the ids its native picker offers.

Agent definitions are deliberately absent: the tool does not model, read, or write them. A per-agent model pin is orthogonal user state that routes through the same per-harness route.

The invariant folds in *all* route coherence, so every reachable state — not merely states an operation happened to build — is checked by the model checker:

Config

$$
\begin{array}{l}
configured : \mathbb{P}\ Source \\
routeVia : Harness \pfun Source \\
model : Harness \pfun ModelId \\
hauth : Harness \pfun AuthMode \\
override : \mathbb{P}\ Harness \\
aliases : Tier \pfun ModelId \\
shown : Harness \pfun \mathbb{P}\ ModelId \\
\hline
\dom model = \dom routeVia \\
\dom hauth = \dom routeVia \\
\ran routeVia \subseteq configured \\
override \subseteq canonicalizes \cap \dom routeVia \\
\dom shown = showsPicker \cap \dom routeVia \\
(\forall h : \dom routeVia \bullet \\
\quad speaks\ h \in srcProtocols\ (routeVia\ h) \land \\
\quad providerOf\ (model\ h) \in srcServes\ (routeVia\ h) \land \\
\quad hauth\ h \in srcAuth\ (routeVia\ h) \land \\
\quad (h \in canonicalizes \Rightarrow h \in override) \land \\
\quad (hauth\ h = subauth \Rightarrow anthropic \in srcProtocols\ (routeVia\ h))) \\
(claudecode \in \dom routeVia \Rightarrow \\
\quad (\dom aliases = \{opus, sonnet, haiku, fable\} \land \\
\quad (\forall t : \dom aliases \bullet \\
\quad\quad providerOf\ (aliases\ t) \in srcServes\ (routeVia\ claudecode)))) \\
(claudecode \notin \dom routeVia \Rightarrow aliases = \varnothing) \\
(\forall h : \dom shown \bullet \\
\quad (\forall m : shown\ h \bullet \\
\quad\quad m \in matchesPattern \land providerOf\ m \in srcServes\ (routeVia\ h)))
\end{array}
$$

6 Initial state

Never configured: no routed harness, no override, no aliases, no picker lists.

Init

$Config'$

$routeVia' = \varnothing$
$model' = \varnothing$
$hauth' = \varnothing$
$override' = \varnothing$
$aliases' = \varnothing$
$shown' = \varnothing$

7 The core guarantees

Because coherence lives in *Config*, these schemas *name* the guarantees for the reader and for targeted checking; they are not the only thing enforcing them. Each is falsifiable: a state violating it is not a legal *Config*.

Default routability (per harness). Every routed harness's default model reaches its source's providers — the operator's pick is served, per harness, by the source the operator chose. This is about the *selected* model, not a definitional set, so it has teeth: a model whose provider the source does not serve violates it.

DefaultRoutable

$Config$

$\forall\, h : \mathrm{dom}\, routeVia \bullet$
$\quad providerOf\ (model\ h) \in srcServes\ (routeVia\ h)$

Protocol coherence. Every routed harness's source speaks that harness's protocol — Claude Code only ever routes through an Anthropic-capable source, the OpenAI harnesses through OpenAI-capable ones.

ProtocolCoherent

$Config$

$\forall\, h : \mathrm{dom}\, routeVia \bullet speaks\ h \in srcProtocols\ (routeVia\ h)$

Claude routing-preservation. When Claude Code is routed, its override is written (so its canonicalized id is restored on the wire) and all four tier knobs are set to ids routable on its source (so no model-less tier spawn mis-routes). The "all four set" requirement is what stops a routed-but-empty alias state passing as correct.

ClaudeRoutingPreserved

Config

$$
\begin{aligned}
&\mathit{claudecode} \in \operatorname{dom} \mathit{routeVia} \Rightarrow \\
&\quad (\mathit{claudecode} \in \mathit{override} \land \\
&\quad \operatorname{dom} \mathit{aliases} = \{\mathit{opus}, \mathit{sonnet}, \mathit{haiku}, \mathit{fable}\} \land \\
&\quad (\forall\, t : \operatorname{dom} \mathit{aliases} \bullet \\
&\qquad \mathit{providerOf}\ (\mathit{aliases}\ t) \in \mathit{srcServes}\ (\mathit{routeVia}\ \mathit{claudecode})))
\end{aligned}
$$

Picker soundness. A native picker never offers an unroutable id: every shown id matches the pattern and its provider is served by that harness's source. The picker is a view ($\mathit{shown}\ h$ need not contain, nor be contained by, any particular model — the operator may route an id the picker does not show). An *empty* picker view ($\mathit{shown}\ h = \varnothing$) is a legal, intentional state (e.g. immediately after routing, before a pattern is applied); the guarantee is "every id the picker shows is routable," which an empty view satisfies correctly — it withholds nothing unroutable because it shows nothing.

PickerSound

Config

$$
\begin{aligned}
&\forall\, h : \operatorname{dom} \mathit{shown} \bullet \forall\, m : \mathit{shown}\ h \bullet \\
&\quad m \in \mathit{matchesPattern} \land \mathit{providerOf}\ m \in \mathit{srcServes}\ (\mathit{routeVia}\ h)
\end{aligned}
$$

The forbidden failure. A routed harness whose default's provider its source does not serve, a canonicalizing harness with no override, or Claude Code routed with a tier knob unset or unroutable — any request that reaches no valid upstream. Not a legal *Config*; named as the predicate the invariant excludes.

SilentMisroute

$$
\begin{aligned}
&\mathit{configured} : \mathbb{P}\ \mathit{Source} \\
&\mathit{routeVia} : \mathit{Harness} \pfun \mathit{Source} \\
&\mathit{model} : \mathit{Harness} \pfun \mathit{ModelId} \\
&\mathit{hauth} : \mathit{Harness} \pfun \mathit{AuthMode} \\
&\mathit{override} : \mathbb{P}\ \mathit{Harness} \\
&\mathit{aliases} : \mathit{Tier} \pfun \mathit{ModelId} \\
&\mathit{shown} : \mathit{Harness} \pfun \mathbb{P}\ \mathit{ModelId}
\end{aligned}
$$

$$
\begin{aligned}
&\operatorname{dom} \mathit{model} = \operatorname{dom} \mathit{routeVia} \\
&((\exists\, h : \operatorname{dom} \mathit{routeVia} \bullet \\
&\quad \mathit{providerOf}\ (\mathit{model}\ h) \notin \mathit{srcServes}\ (\mathit{routeVia}\ h)) \lor \\
&\quad (\exists\, h : \mathit{canonicalizes} \cap \operatorname{dom} \mathit{routeVia} \bullet h \notin \mathit{override}) \lor \\
&\quad (\mathit{claudecode} \in \operatorname{dom} \mathit{routeVia} \land \\
&\qquad \operatorname{dom} \mathit{aliases} \neq \{\mathit{opus}, \mathit{sonnet}, \mathit{haiku}, \mathit{fable}\}))
\end{aligned}
$$

8 Operation: `route` — the operator picks a route

The one selecting operation. The operator names a set of harnesses $hs?$, a source $s?$, a default model $m?$, an auth mode $a?$, and (when Claude Code is in $hs?$) the tier ids $tiers?$. It routes exactly those harnesses through $s?$, last-write-wins per harness — so a later `route` of a different source for a harness is the operator changing their preferred route, with no precedence rule and no overlap ban. Preconditions (all enforced *in the operation*, so the shared effect alone establishes $Config'$): $s?$ is configured; $s?$ speaks every chosen harness's protocol; $m?$'s provider is served by $s?$; $a?$ is accepted by $s?$; *subauth* only where $s?$ is Anthropic-capable; and if Claude Code is chosen, all four tiers are supplied and routable on $s?$.

RouteOk

$$
\begin{array}{l}
\Delta Config \\
hs? : \mathbb{P}\ Harness \\
s? : Source \\
m? : ModelId \\
a? : AuthMode \\
tiers? : Tier \pfun ModelId \\
\hline
hs? \neq \emptyset \\
s? \in configured \\
(\forall\, h : hs? \bullet speaks\ h \in srcProtocols\ s?) \\
providerOf\ m? \in srcServes\ s? \\
a? \in srcAuth\ s? \\
(a? = subauth \Rightarrow anthropic \in srcProtocols\ s?) \\
(claudecode \in hs? \Rightarrow \\
\quad (\mathrm{dom}\ tiers? = \{opus, sonnet, haiku, fable\} \land \\
\quad (\forall\, t : \mathrm{dom}\ tiers? \bullet providerOf\ (tiers?\ t) \in srcServes\ s?))) \\
configured' = configured \\
routeVia' = routeVia \oplus \{h : hs? \bullet h \mapsto s?\} \\
model' = model \oplus \{h : hs? \bullet h \mapsto m?\} \\
hauth' = hauth \oplus \{h : hs? \bullet h \mapsto a?\} \\
override' = override \cup (canonicalizes \cap hs?) \\
(claudecode \in hs? \Rightarrow aliases' = tiers?) \\
(claudecode \notin hs? \Rightarrow aliases' = aliases) \\
shown' = (showsPicker \cap \mathrm{dom}\ routeVia') \lhd shown \\
\quad \oplus \{h : showsPicker \cap hs? \bullet h \mapsto \emptyset\}
\end{array}
$$

The refusals, each a no-op with a distinct code: source not configured; source cannot carry a chosen harness's protocol; the default's provider not served; the auth mode not accepted; a *subauth* request to a non-Anthropic source; or Claude Code chosen without all four routable tiers.

RouteRefused

$\Xi Config$
$hs? : \mathbb{P}\ Harness$
$s? : Source$
$m? : ModelId$
$a? : AuthMode$
$tiers? : Tier \pfun ModelId$
$err! : \mathbb{Z}$

$(s? \notin configured \lor$
$\quad (\exists h : hs? \bullet speaks\ h \notin srcProtocols\ s?) \lor$
$\quad providerOf\ m? \notin srcServes\ s? \lor$
$\quad a? \notin srcAuth\ s? \lor$
$\quad (a? = subauth \land anthropic \notin srcProtocols\ s?) \lor$
$\quad (claudecode \in hs? \land$
$\quad\quad \neg\ (\mathrm{dom}\ tiers? = \{opus, sonnet, haiku, fable\} \land$
$\quad\quad\quad (\forall t : \mathrm{dom}\ tiers? \bullet providerOf\ (tiers?\ t) \in srcServes\ s?))))$
$err! = 1$

$Route \mathrel{\widehat=} RouteOk \lor RouteRefused$

9 Operation: `unroute` (direct)

Return a set of harnesses to their vendor defaults: drop them from every managed map. Clearing Claude Code also clears the aliases (the invariant requires $aliases = \varnothing$ when Claude Code is unrouted).

Unroute

$\Delta Config$
$hs? : \mathbb{P}\ Harness$

$configured' = configured$
$routeVia' = hs? \lhd routeVia$
$model' = hs? \lhd model$
$hauth' = hs? \lhd hauth$
$override' = override \setminus hs?$
$(claudecode \in hs? \Rightarrow aliases' = \varnothing)$
$(claudecode \notin hs? \Rightarrow aliases' = aliases)$
$shown' = hs? \lhd shown$

10 Read-only operations: `status`, `list`, `verify`

Status reports the per-harness route and default. *List* reports, for a source, the providers it serves. *Verify* is the probe: $ok! = 1$ exactly when the configuration is

a legal *Config* in which every routed harness is fully correct — default routable, protocol matched, auth accepted, canonicalizing override present, and (if Claude Code routed) all four tiers routable. Because *Config* already enforces these, *Verify* cannot pass a mis-routed state.

Status

$$\Xi Config$$
$$route! : Harness \pfun Source$$
$$dflt! : Harness \pfun ModelId$$
$$route! = routeVia$$
$$dflt! = model$$

List

$$\Xi Config$$
$$s? : Source$$
$$provs! : \mathbb{P}\ Provider$$
$$provs! = srcServes\ s?$$

VerifyPass

$$\Xi Config$$
$$ok! : \mathbb{Z}$$
$$\begin{lparray}(\forall\, h : \mathrm{dom}\ routeVia \bullet \\ \quad providerOf\ (model\ h) \in srcServes\ (routeVia\ h) \land \\ \quad speaks\ h \in srcProtocols\ (routeVia\ h) \land \\ \quad hauth\ h \in srcAuth\ (routeVia\ h) \land \\ \quad (h \in canonicalizes \Rightarrow h \in override)) \\ (claudecode \in \mathrm{dom}\ routeVia \Rightarrow \\ \quad \mathrm{dom}\ aliases = \{opus, sonnet, haiku, fable\}) \\ ok! = 1\end{lparray}$$

VerifyFail

$$\Xi Config$$
$$ok! : \mathbb{Z}$$
$$\begin{lparray}((\exists\, h : \mathrm{dom}\ routeVia \bullet \\ \quad providerOf\ (model\ h) \notin srcServes\ (routeVia\ h) \lor \\ \quad speaks\ h \notin srcProtocols\ (routeVia\ h) \lor \\ \quad hauth\ h \notin srcAuth\ (routeVia\ h) \lor \\ \quad (h \in canonicalizes \land h \notin override)) \lor \\ \quad (claudecode \in \mathrm{dom}\ routeVia \land \\ \qquad \mathrm{dom}\ aliases \neq \{opus, sonnet, haiku, fable\})) \\ ok! = 0\end{lparray}$$

$$Verify \mathrel{\widehat=} VerifyPass \lor VerifyFail$$

11 The system invariant and the apply obligation

SystemOK restates, as a single predicate, that every routed harness is fully correct — it is exactly the body *Config* enforces, named for checking.

SystemOK

Config

$$\begin{array}{l}
(\forall\, h : \mathrm{dom}\, routeVia \bullet \\
\quad providerOf\,(model\, h) \in srcServes\,(routeVia\, h) \land \\
\quad speaks\, h \in srcProtocols\,(routeVia\, h) \land \\
\quad hauth\, h \in srcAuth\,(routeVia\, h) \land \\
\quad (h \in canonicalizes \Rightarrow h \in override)) \land \\
(claudecode \in \mathrm{dom}\, routeVia \Rightarrow \\
\quad \mathrm{dom}\, aliases = \{opus, sonnet, haiku, fable\})
\end{array}$$

The real obligation, with no smuggled hypothesis: every harness *RouteOk* touches ends routable on the source the operator chose, established by *RouteOk*'s own preconditions and effect alone. ProB checks it via `-cbc_assertions`.

Theorem (RouteEstablishesRoutability).

$$\begin{array}{l}
\forall\, \Delta Config;\ hs? : \mathbb{P}\, Harness;\ s? : Source;\ m? : ModelId; \\
\quad a? : AuthMode;\ tiers? : Tier \pfun ModelId \bullet \\
\quad\quad RouteOk \Rightarrow \\
\quad\quad\quad (\forall\, h : hs? \bullet \\
\quad\quad\quad\quad providerOf\,(model'\, h) \in srcServes\,(routeVia'\, h))
\end{array}$$

12 On-disk configuration: what the tool writes, per source

The tool writes, for each routed harness, only its *route* (base URL + auth, pointing at the chosen source's endpoint) and its *default model* — plus, for Claude Code, the canonicalization override and the four tier knobs, and, for pi/opencode, the native picker list. It never writes an agent definition.

Claude Code. `.envrc.local` + `.claude/settings.local.json`. Route: `ANTHROPIC_BASE_URL`, `ANTHROPIC_AUTH_TOKEN`. Default: `ANTHROPIC_MODEL` (verbatim, no [1*m*] marker — it breaks the override-key match). Override: `modelOverrides`. Tier knobs: `ANTHROPIC_DEFAULT_{OPUS,SONNET,HAIKU,FABLE}_MODEL` and `ANTHROPIC_SMALL_FAST_MODE`. No picker. A `.claude/agents/*.md` with its own `model:` is a user agent definition — out of scope.

codex. `config.toml`. Route: the matching `[model_providers.*]` table. Default: `model` and the single `[agents] agent_default_subagent_model`. No native picker list, no per-agent files.

pi. `~/.pi/agent/{settings,models}.json`. Route: the source provider block + `defaultProvider`. Default: `defaultModel`. Picker: the source provider's `models` array (*shown*) — the pattern-matched, routable subset. A `.pi/agents/*.md` frontmatter `model:` is out of scope.

opencode. `opencode.json`. Route: the `provider.<name>.options` block. Default: global `model` + `small_model`. Picker: the `provider.<name>.models` map (*shown*). An `agent.<name>.model` is out of scope.

The uniform rule: the tool writes the route, the default, and (for picker harnesses) the picker view; it never writes an agent's model. A per-agent model the user pinned still routes, because *routeVia* is the one shared route for that harness and the source serves whatever provider the pinned id names — or it does not, in which case that is the user's pin to fix, not the tool's.

13 Sources as conforming instances

Bifrost is one source: it exposes both protocols and both auth modes and serves many providers by fanning out on the model-id prefix — so a single Bifrost source can carry all four harnesses, the single-source operator's case. A direct source (direct OpenAI, direct Anthropic) typically exposes one protocol and serves one provider; an aggregator (OpenRouter) may expose both protocols and serve several providers without being a gateway. The multi-source operator composes these: Claude Code routed through an Anthropic-capable source, the OpenAI harnesses through whichever source they prefer. Nothing in the spec is true only of a gateway; a re-implementation swaps or mixes sources without changing the guarantees.

A source that can carry the whole fleet is one exposing both protocols; the spec does not require any single such source to exist (the multi-source operator has none), only that each routed harness's source speaks that harness's protocol.

$$\begin{array}{|l} \mathit{fleetCapable} : \mathbb{P}\ \mathit{Source} \\ \hline \mathit{fleetCapable} = \{s : \mathit{Source} \mid \\ \qquad \mathit{anthropic} \in \mathit{srcProtocols}\ s \land \mathit{openai} \in \mathit{srcProtocols}\ s\} \end{array}$$

14 Two real-world constraints: compatibility and credential terms

Compatibility (support, not permission). Anthropic does not support routing Claude Code to non-Claude models. This is a support and compatibility statement, not a terms violation: with a non-Anthropic default on the Claude Code route, tool use, extended thinking, and prompt caching are unvalidated and degrade per model, so routability is necessary but not sufficient. Separately,

MCP tool search and fine-grained tool streaming silently turn off whenever Claude Code is routed through any source other than Anthropic-direct; both are recoverable via environment variables. These are side obligations of a routed Claude Code, named here rather than encoded in the invariant.

Credential terms (enforced). The licensing line runs along the credential, not the model: a Claude Pro/Max subscription is licensed for Anthropic's own clients (the Claude apps and Claude Code), so spending it on a third-party client is the violation — for any model. Here *subauth* is the subscription-OAuth mode, and *Config* plus *RouteOk* permit it only where the route is Anthropic-capable, which for a non-Claude harness would require that harness to route through an Anthropic source it does not speak to — excluded by protocol coherence. *vkauth* (per-host keys billing the upstream) is clean everywhere.